HDOC Day-46

Question

Write a C program to read the size and elements of a variable-length integer array (which may contain duplicates). Pass the array by address to **Bubble_sort()** to sort it in ascending order. Then pass the sorted array using pointers to **occurrence()**. Read a target integer and display the value and index of its first and last occurrences. If the target is absent, display -1, -1.

C Program

```
#include <stdio.h>

void Bubble_sort(int *nums, int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (*(nums + j) > *(nums + j + 1)) {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *nums, int n) {
    int target, first = -1, last = -1, i;
    printf("Enter target: ");
    scanf("%d", &target);

    for (i = 0; i < n; i++) {
        if (*(nums + i) == target) {
            if (first == -1)
                first = i;
            last = i;
        }
    }

    if (first == -1) {
        printf("-1, -1\n");
    } else {
        printf("First occurrence: %d at index %d\n",
               *(nums + first), first);
        printf("Last occurrence: %d at index %d\n",
               *(nums + last), last);
    }
}

int main(void) {
    int n, i;
    printf("Enter array size: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid size\n");
        return 1;
    }

    int nums[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    Bubble_sort(nums, n);
```

```
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\n");

    occurrence(nums, n);
    return 0;
}
```

Sample Output

```
Enter array size: 6
Enter 6 elements: 5 7 7 8 8 10
Sorted array: 5 7 7 8 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 4
```

Explanation

Bubble_sort() compares adjacent elements and swaps them when needed. The occurrence() function checks the sorted array using pointer arithmetic and records the first and last indices of the target. Array indices start at 0.